courseCS.WordPress.com

Lesson-4-Loops

Download Lesson

Previous Lesson Next Lesson

A loop in programming is a control structure that allows a set of instructions to be repeated multiple times based on a specified condition. Loops are used to automate repetitive tasks and efficiently process a sequence of data or perform a block of code multiple times. They are an essential part of almost all programming languages.

4.1. Types of Loop

JavaScript supports different kinds of loops.

1. `for` – loops through a block of code a number of times
2. `for/in` – loops through the properties of an object
3. `for/of` – loops through the values of an iterable object
4. `while` – loops through a block of code while a specified condition is true
5. `do/while` – also loops through a block of code while a specified condition is true

Loop is mostly used with arrays, therefore, we explained loop on based on arrays in next sections.

4.1.1. Loop “for”

The `for` statement creates a loop with 3 optional expressions.

- **Expression 1** is executed (one time) before the execution of the code block.
- **Expression 2** defines the condition for executing the code block.
- **Expression 3** is executed (every time) after the code block has been executed.

```
for (expression 1; expression 2; expression 3) {
  // code block to be executed
}
```

Example Code 1:

```
//const is used to declared array
const cars = ["Saab", "Volvo", "BMW"]

//displaying each element by using loop
for(let i=0; i<cars.length; i++)
console.log(cars[i])
```

4.1.1.1 Expression 1 in “for” Loop

Normally you will use expression 1 to initialize the variable used in the loop (let i = 0). This is not always the case. JavaScript doesn’t care. Expression 1 is optional. You can initiate many values in expression 1 (separated by comma).

Example Code 2:

```
const cars = ["Saab", "Volvo", "BMW"]

//displaying each elment by using loop
for(let i=0, length=cars.length; i<length; i++)
console.log(cars[i])
```

And you can omit expression 1 (like when your values are set before the loop starts) as you can see in below code.

Example Code 3:

```
const cars = ["Saab", "Volvo", "BMW"]

//initialize values for loop
let i=0, length=cars.length

//no expression 1
for(; i<length; i++)
   console.log(cars[i])
```

Example Code 4: If you don’t know about length of a array, then you can use either “for in” or “for of” loop. There is another method by using “for” loop that is shown in following code. Output is same just like above.

```
const cars = ["Saab", "Volvo", "BMW"];
let i=0
for (;cars[i];) {
  console.log(cars[i])
  i++
}
```

4.1.1.2 Expression 2 in "for" Loop

Often expression 2 is used to evaluate the condition of the initial variable. This is not always the case. JavaScript doesn't care. Expression 2 is also optional. If expression 2 returns true, the loop will start over again. If it returns false, the loop will end.

Note: If you omit expression 2, you must provide a **break** inside the loop. Otherwise the loop will never end. This will crash your browser. Read about breaks in a later chapter of this tutorial.

Example Code 5:

```
let j=0, v=10
for(;;j++){
    console.log(j)
    if(j==10)
        break
}
```

4.1.1.3 Expression 3 in "for" Loop

Often expression 3 increments the value of the initial variable. This is not always the case. JavaScript doesn't care. Expression 3 is optional. Expression 3 can do anything like negative increment (i–), positive increment (i = i + 15), or anything else. Expression 3 can also be omitted (like when you increment your values inside the loop).

Example Code 6:

```
let j=5, v=20
for(;;){
    console.log(j)
    if(j==10)
        break
    j++
}
```

4.1.2. Loop "for in"

The JavaScript `for in` statement loops through the properties of an Object (object, array and date). Syntax for this loop is given below.

```
for (key in object) {
  // code block to be executed
}
```

Note: In case of `for...in`, the "key" will contain indexes (named-index in case of object and numbered-indexes in case of array)

Importance of loop "for in" over "for" is that you don't need to know length of total properties/elements of objects.

Example Code 7: In following code, an object "person" is declared by using `const` keyword and then all properties of the objects are displayed by using the loop.

It is noted that `let` is used declare "x". It is note necessary to use `let`. You can use either `const` or `let` according to your requirement. The `const` is necessary to use at the time of declaration of the object.

```
const person = {fname:"John", lname:"Doe", age:25}

for (let x in person) {
  console.log(x) //only property's key would be displayed
}
```

Note: Only enumerable properties are included when using a `for...in` loop to iterate over an object. While, non-enumerable properties would not be included. It would be discussed in detail in lesson of object.

Example Code 8: Following code is just like above but in this case "for in" loop is used for array.

```
const numbers = [45, 4, 9, 16, 25];

for (let x in numbers) {
  console.log(x) //only array indexes (0,1,2,3,4,5) would be displayed
}
```

Note: Do not use **for in** over an Array if you want to access array's element in an order. By using this loop, array values may not be accessed in the order you expect. If element's order is important, then it is better to use a **for** loop or a **for of** loop.

4.1.3. Loop "for of"

The JavaScript `for of` statement loops through the values of an iterable object. It lets you loop over iterable data structures such as Arrays, Strings, Maps, NodeLists, and more. Syntax for this loop is given below.

Note: Importance of loop "for of" over "for" is that you have few conditions related to loop.

Note: Importance of loop "for of" over "for in" is that it is used for many iterable objects but "for in" is only useful for objects and arrays.

```
for (key of iterable) {
  // code block to be executed
}
```

Note: In case of for...of, the "key" will contain element value instead of index number. You can use for...in in case of both array and object but for...of is only useful for array.

Example Code 9: In following code, an array "numbers" is declared by using `const` keyword and then all properties of the objects are displayed by using the loop.

Note: If you use "for in" loop here, then result of both codes would be same.

```
const numbers = [45, 4, 9, 16, 25];

for (let x of numbers) {
  console.log(x)
}
```

Example Code 10: In following code, a string “courseName” is used with loop.

Note: If you use “for in” loop here, then result of both codes would not be same. Error will be generated but “for in” loop will treat string/any-other-iterable-objects differently.

```
let courseName = "web development"

for (let x of courseName) {
  console.log(x)// all letter of above string will be displayed in one column
}
```

4.1.4. Loop “while”

The `while` loop loops through a block of code as long as a specified condition is true.

Example Code 11: In following code, body of the loop would be executed as long as a variable (i) is less than 10.

```
let i=0
while (i < 10) {
    console.log("value of i: ", i)
    i++;
  }
```

Note: If you forget to increase the variable used in the loop-body, the loop will never end. This will crash your browser.

4.1.5. Loop “do while”

The `do while` loop is a variant of the while loop. This loop will execute the code block once, before checking if the condition is true, then it will repeat the loop as long as the condition is true. Syntax for this loop is given below

```
do {
  // code block to be executed
}
while (condition)
```

Example Code 12: The example below uses a `do while` loop. The loop will always be executed at least once, even if the condition is false, because the code block is executed before the condition is tested.

This code is just line code 14, but we just use do while loop.

```
let i=0
 do {
    console.log("value of i: ", i)
    i++;
  }
while (i < 10)
```

Note: If you forget to increase the variable used in the loop-body, the loop will never end. This will crash your browser.

Example Code 13: Following code will iterate each element by using `do while` loop without calculating length of the array. Similar code can be write by using `while` loop. Such type of code is also discussed in top of `for` loop.

```
const cars = ["Saab", "Volvo", "BMW"];
let i=0
do{
  console.log(cars[i])
  i++
}
while(cars[i])
```

4.2. Scope of Variable Used in Loop

If you used `var` type variable (having same name) for both inside and outside loop , then it would be declared generally. In other words, same memory is used for both outside and inside loop. But in this case, you can redeclare the variable (normally, it is not allowed) inside loop.

Example Code 14: In this code, same memory is used for `var` type same-variable in case of inside and outside of loop. It is noted that variable can be redeclared in side loop.

```
var i=5
for(var i=0; i<10; i++){
    console.log(i)  //it will display values from 0 to 9
}
console.log(i)  //it will display value 10 for variable i
```

But if you used `let` type, then separate memory is allocated for variables outside and inside loop. See following code.

Example Code 15:

```
let i=5
for(let i=0; i<10; i++){
    console.log(i)  //it will display values from 0 to 9
}
console.log(i)  //it will display value 5 for variable i
```

Note: You cannot change type of declaration inside loop. See example code 8 and 9. Both codes will generate errors for redeclaration.

Example Code 16: Error of redeclaration for line 1 and 2

```
1  let i=5
2  for(var i=0; i<10; i++){
3      console.log(i)
4  }
5  console.log(i)
```

Example Code 17: Error of redeclaration for line 1 and 2

```
1  var i=5
2  for(let i=0; i<10; i++){
3      console.log(i)
4  }
5  console.log(i)
```

Exercises

Solve following exercises to enhance your learnt concepts related to this lesson.

1. Write a for loop that counts from 1 to 10 and displays each number in the console.
2. Create an array of your favorite fruits. Use a for loop to iterate through the array and display each fruit in the console.
3. Write a while loop that counts from 1 to 5 and displays each number in the console.
4. Create an array of numbers from 10 to 1. Use a for loop to iterate through the array in reverse order and display each number.
5. Write a for loop that counts from 1 to 20. For each number, check if it's even or odd and display a message in the console.

▼ Solution for Exercise 1

```
1  for (let i = 1; i <= 10; i++) {
2    console.log(i);
3  }
```

▼ Solution for Exercise 2

```
const favoriteFruits = ['Apple', 'Banana', 'Orange', 'Grapes', 'Strawberry'];

for (let i = 0; i < favoriteFruits.length; i++) {
  console.log(favoriteFruits[i]);
}
```

▼ Solution for Exercise 3

```
let counter = 1;

while (counter <= 5) {
  console.log(counter);
  counter++;
}
```

▼ Solution for Exercise 4

```
const numbersArray = [10, 9, 8, 7, 6, 5, 4, 3, 2, 1];

// Iterating through the array in reverse order and displaying each number
for (let i = numbersArray.length - 1; i >= 0; i--) {
  console.log(numbersArray[i]);
}
```

▼ Solution for Exercise 5

```
for (let i = 1; i <= 20; i++) {
  if (i % 2 === 0) {
    console.log(`${i} is even.`); /* backticks are used */
  } else {
    console.log(i+" is odd"); /* concatination is used */
  }
}
```

Previous Lesson **Next Lesson**

.

.

.

.

.

.